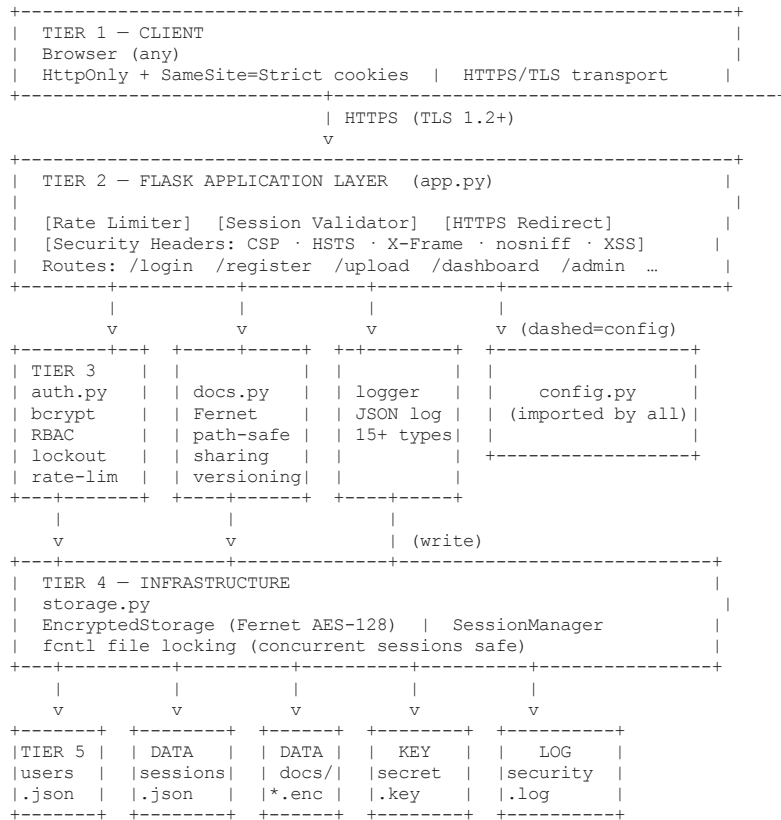
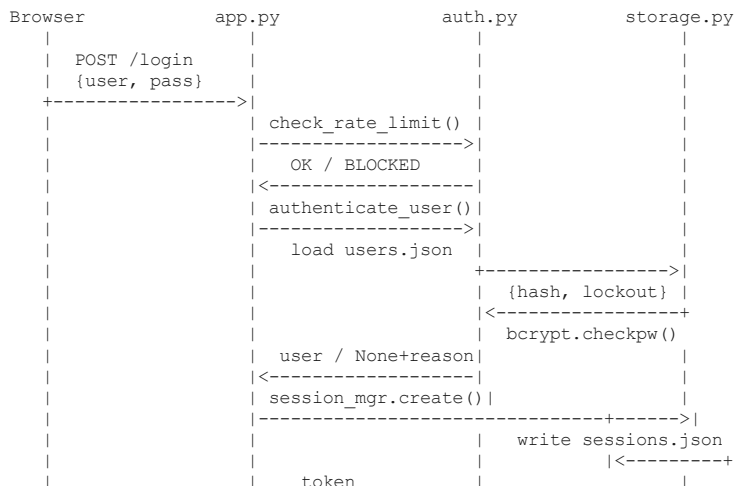


SecureDocs is structured as a five-tier layered architecture. Each tier has a single responsibility, and communication flows strictly downward — no tier reaches across to a peer tier or upward to a caller. The diagram below represents this layout:





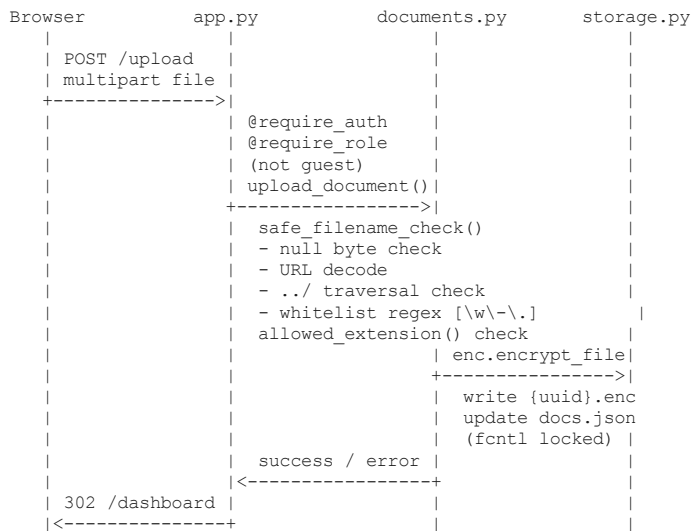
```

| 302 + Set-Cookie |<-----+-----+
| HttpOnly;Secure | | |
|<-----+ | |

```

Security controls applied in this flow: IP-based rate limiting (10 req/min), bcrypt cost-factor 12 password verification, account lockout after 5 failures (15 min), 256-bit session token generation, HttpOnly + SameSite=Strict + Secure cookie attributes.

DFD 2 — Document Upload & Encryption



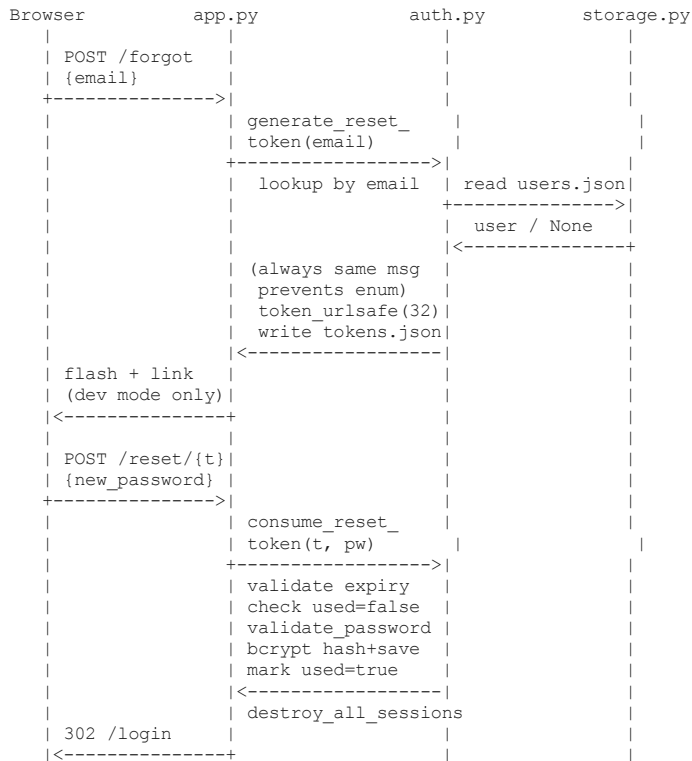
Security controls applied in this flow: RBAC role check (guests cannot upload), four-stage filename sanitisation, MIME/extension whitelist (pdf/txt/docx/png/jpg), 16 MB content length cap (Flask MAX_CONTENT_LENGTH), Fernet AES-128 encryption before disk write, fcntl exclusive file lock to prevent concurrent overwrites.

DFD 3 — Document Sharing & Access Control



Permission model enforced at every access: owners can read/write/delete/share; editors can read and write; viewers can only read; admins bypass all document-level checks. Every access attempt is logged to security.log with event type DATA_ACCESS or ACCESS_DENIED.

DFD 4 — Password Reset Flow



Security controls: tokens are 256-bit cryptographically random (secrets.token_urlsafe), expire after 1 hour, are single-use (marked used=true on consumption), and cannot be replayed. The forgot-password endpoint always returns the same message regardless of whether the email exists, preventing user enumeration. All active sessions are destroyed on password change.

A.3 Component Descriptions

app.py — Flask Application Coordinator

The central entry point and request coordinator for the application. It does not contain business logic — its role is to receive HTTP requests, apply middleware, delegate to the appropriate module, and return HTTP responses.

Responsibility	Detail
Before-request hooks	load_session() resolves the session cookie to a user object via SessionManager. force_https() redirects HTTP to HTTPS in production. check_rate_limit() is called inline on the /login route.
After-request hooks	set_security_headers() appends all seven required headers to every response: CSP, X-Frame-Options, X-Content-Type-Options, X-XSS-Protection, Referrer-Policy, Permissions-Policy, Strict-Transport-Security.
Route protection	Routes use @require_auth and @require_role('admin') decorators defined in auth.py. Guest routes abort(403) if the user's role is 'guest'.
Error handling	Handlers for 403, 404, 413, and 500 return generic messages to the user. Internal details are only written to app.logger — never exposed in the response body.
Jinja2 filter	A custom datetime filter converts Unix timestamps to human-readable strings (e.g. Apr 20, 2026 02:30 PM) without exposing raw floats in templates.

auth.py — Authentication & Authorisation Module

Owens all identity-related logic. No route handler performs authentication directly — app.py delegates entirely to the functions and decorators defined here.

Function	Security behaviour
validate_username()	Whitelist regex <code>[w]{3,20}</code> — 3 to 20 alphanumeric/underscore characters. Rejects all special characters, spaces, and path-traversal sequences.
validate_password()	Enforces: minimum 12 characters; at least one uppercase, lowercase, digit, and special character (<code>!@#\$%^&*</code>). Returns structured (bool, message) tuple.
register_user()	Validates all fields, checks for duplicate username and email, hashes the password with bcrypt cost factor 12, and persists the user record. Default role is 'user'.
authenticate_user()	Checks logout status first, then calls <code>bcrypt.checkpw()</code> . On failure, increments <code>failed_attempts</code> and locks the account at 5. Returns identical error messages for unknown user vs wrong password (timing-safe).
check_rate_limit()	Maintains an in-memory dict keyed by IP. Timestamps older than <code>RATE_LIMIT_WINDOW</code> (60s) are pruned on each call. Blocks IPs exceeding <code>RATE_LIMIT_MAX</code> (10) attempts per window.
generate_reset_token()	Creates a 256-bit URL-safe token with a 1-hour expiry. Purges any existing unused token for the same user before writing the new one. Always returns a generic 'if registered' message.
consume_reset_token()	Validates token existence, expiry, and <code>used=false</code> . On success: hashes the new password, resets logout fields, marks the token <code>used=true</code> (not deleted — allows replay detection).
require_auth / require_role	Flask route decorators using <code>functools.wraps</code> . <code>require_auth</code> checks <code>g.user</code> is set; <code>require_role(*roles)</code> calls <code>abort(403)</code> if the user's role is not in the allowed list.

documents.py — Document Management Module

Handles the full lifecycle of uploaded documents: validation, encryption, storage, sharing, and deletion. Every write operation goes through `EncryptedStorage` in `storage.py` and every read checks the permission model.

Function	Security behaviour
safe_filename_check()	Four-stage pipeline: (1) reject null bytes; (2) URL-decode to catch <code>%2F%2E%2E</code> encoded traversal; (3) reject any path segment composed entirely of dots (<code>., ..., ...</code>); (4) apply <code>werkzeug.utils.secure_filename</code> ; (5) whitelist regex <code>[w\-\.\.]+</code> .
upload_document()	Validates filename and extension, saves to a temp path, calls <code>EncryptedStorage.encrypt_file()</code> to write the Fernet-encrypted <code>.enc</code> file, then removes the temp file. Document metadata (shares, versions, owner) is written to <code>documents.json</code> under <code>fcntl</code> exclusive lock.
can_read / can_write / can_delete	Pure functions implementing the permission matrix. Admin bypasses all checks. Owner can do everything. Editor can read and write. Viewer can only read. No role other than owner or admin can delete.
download_document()	Calls <code>can_read()</code> before any I/O. Performs a secondary path traversal guard using <code>os.path.abspath()</code> to confirm the resolved path is inside <code>Config.UPLOAD_DIR</code> before calling <code>decrypt_file()</code> .
share_document()	Only proceeds if <code>can_write()</code> returns True (owner or admin or editor). Writes the updated shares dict to <code>documents.json</code> under <code>fcntl</code> lock to prevent concurrent share-grant/revoke races.

storage.py — Encrypted Storage & Session Manager

The only module that performs disk I/O for sensitive data. All other modules call into `storage.py` rather than writing files directly. This centralises the encryption and locking concerns.

EncryptedStorage

- Uses the cryptography library's Fernet class, which provides AES-128-CBC with HMAC-SHA256 authentication.
- On first run, generates a 256-bit key and writes it to `data/secret.key` with `chmod 600` (owner read-only).

- `save(path, data)` serialises Python dicts/lists to JSON, then encrypts the bytes before writing to disk. The on-disk format is opaque binary — even reading the raw file reveals nothing about the content.
- `encrypt_file(src, dest)` reads a raw uploaded file and writes it to a new `.enc` file using the same Fernet cipher, then the caller removes the unencrypted temp file.

SessionManager

- Tokens are generated with `secrets.token_urlsafe(32)`, producing 256 bits of cryptographically random data encoded as URL-safe base64.
- All writes to `sessions.json` are wrapped in an `fcntl.LOCK_EX` exclusive file lock to prevent concurrent login/logout operations from overwriting each other's session entries.
- `validate()` performs a read-only check and only acquires the write lock if the session has expired (to remove it). The `last_activity` timestamp is refreshed at most once per 60 seconds via `touch()` to avoid hammering the file on every page load.
- `destroy_all_for_user(user_id)` invalidates every session belonging to a user — called on password change and admin password reset.

logger.py — Security Event Logger

A module-level singleton (`security_log`) that is imported and used by all other modules. Events are written as structured JSON objects, one per line, to `logs/security.log`.

Each log entry contains: timestamp (ISO 8601 UTC), `event_type`, severity (`DEBUG/INFO/WARNING/ERROR/CRITICAL`), `user_id`, `ip_address`, `user_agent`, and a `details` dict with event-specific fields.

Events logged include: `LOGIN_SUCCESS`, `LOGIN_FAILED`, `ACCOUNT_LOCKED`, `LOGOUT`, `REGISTRATION`, `ACCESS_DENIED`, `DATA_ACCESS`, `SESSION_CREATED`, `SESSION_EXPIRED`, `RATE_LIMIT_EXCEEDED`, `SUSPICIOUS_ACTIVITY`, `PASSWORD_CHANGED`, `PASSWORD_RESET_REQUESTED`, `DOCUMENT_SHARED`, `DOCUMENT_DELETED`, `ROLE_CHANGED`, `ADMIN_PASSWORD_RESET`.

config.py — Centralised Configuration

A single Python class with class-level attributes. All tunable security parameters live here so they can be changed in one place without hunting through application code. Sensitive values are read from environment variables with safe defaults for development.

Parameter	Default	Purpose
SESSION_TIMEOUT	1800 s	Inactivity timeout for sessions (30 minutes)
MAX_FAILED_ATTEMPTS	5	Consecutive login failures before account lockout
LOCKOUT_DURATION	900 s	Account lockout duration (15 minutes)
RATE_LIMIT_WINDOW	60 s	Sliding window for login rate limiting
RATE_LIMIT_MAX	10	Maximum login attempts per IP per window
MAX_CONTENT_LENGTH	16 MB	Flask hard cap on request body size
RESET_TOKEN_EXPIRY	3600 s	Password reset token lifetime (1 hour)
ALLOWED_EXTENSIONS	5 types	pdf, txt, docx, png, jpg — upload whitelist

A.4 Technology Stack Justification

Every library and framework in the stack was chosen for a specific security or pedagogical reason. The table below documents the decision rationale for each dependency.

Component	Technology	Justification
Web framework	Flask 3.0	Lightweight WSGI framework with explicit, transparent request handling. Unlike Django, Flask requires the developer to make all security decisions consciously — no implicit CSRF middleware means we implement and understand each control. Jinja2 auto-escaping is enabled by default, preventing XSS in templates.

Password hashing	bcrypt (cost 12)	bcrypt is purpose-built for password storage. Its work factor (cost=12, ~250ms on modern hardware) makes brute-force attacks computationally expensive. Unlike SHA-256 or MD5, bcrypt is deliberately slow and includes a built-in salt. Argon2 is the modern alternative, but bcrypt remains the course-recommended library and is universally supported.
Encryption at rest	Fernet (cryptography)	Fernet provides authenticated symmetric encryption using AES-128-CBC + HMAC-SHA256. It is designed to be misuse-resistant — the high-level API prevents common mistakes like forgetting authentication or reusing IVs. The cryptography library is maintained by the Python Cryptographic Authority and undergoes regular security audits.
Session tokens	secrets module	Python's secrets module uses the operating system CSPRNG (os.urandom) to generate cryptographically strong random values. secrets.token_urlsafe(32) produces 256 bits of entropy encoded as URL-safe base64. The random module must never be used for security tokens as it is not cryptographically secure.
File locking	fcntl (stdlib)	JSON files are not thread/process-safe by default. fcntl.LOCK_EX acquires an exclusive advisory lock at the OS level, preventing concurrent writes from corrupting the file or silently losing data. This was added after discovering a race condition where simultaneous sessions from two browsers would overwrite each other's session records.
Input sanitisation	html.escape + werkzeug	html.escape() is used to sanitise all user-provided strings before they are stored or rendered, preventing stored XSS. Jinja2's auto-escaping provides a second layer of defence at render time. werkzeug.utils.secure_filename() normalises upload filenames, and our safe_filename_check() adds URL-decode and dot-segment rejection on top.
Templating	Jinja2	Bundled with Flask. All template variables are HTML-escaped by default ({{ var }} always safe; {{ var safe }} must be used explicitly and deliberately). This prevents reflected and stored XSS as a default posture rather than an opt-in.
Frontend styling	Bootstrap 5 (CDN)	Loaded from cdn.jsdelivr.net with SRI integrity hashes in the base template. Using a CDN reduces attack surface on the server (no need to serve JS/CSS) while the CSP script-src and style-src directives whitelist only the jsdelivr CDN origin. 'unsafe-inline' is included per the course spec and Bootstrap's requirement for inline styles.
Transport security	TLS (Flask SSL)	In development, a self-signed certificate (4096-bit RSA) is generated with openssl req -x509. In production, the HSTS header (max-age=31536000; includeSubDomains) instructs browsers to refuse all HTTP connections for one year after the first HTTPS visit. The force_https() before_request hook provides a 301 redirect fallback.
Data persistence	JSON files	File-based storage is specified by the course requirements (no database). JSON is human-readable for debugging and auditing. All sensitive JSON files (users, sessions, reset tokens) are written via EncryptedStorage so the raw file content is Fernet ciphertext. documents.json metadata is written under fcntl lock.

Stack Summary

Layer	Technology	Version / Notes
Web framework	Flask	3.0.3
Language	Python	3.12 (type hints, match statements)
Password hashing	bcrypt	4.1.3 — cost factor 12
Symmetric encryption	cryptography (Fernet)	42.0.8 — AES-128-CBC + HMAC-SHA256
JWT (future)	PyJWT	2.8.0 — available for API auth extension
WSGI server	Werkzeug (dev)	3.0.3 — Gunicorn recommended for production
Frontend	Bootstrap 5 + Jinja2	5.3.3 via CDN with CSP whitelist
Testing	unittest (stdlib)	88 tests — runnable with python3 test_security.py
Storage	JSON + fcntl locks	Per course spec — no database required

Section B — Threat Model

This threat model follows the STRIDE methodology (Spoofing, Tampering, Repudiation, Information Disclosure, Denial of Service, Elevation of Privilege) applied to the SecureDocs system. Each asset, threat, and vulnerability is assessed using a likelihood × impact risk score.

B.1 Asset Identification

Assets are grouped into three categories: data assets (information stored or transmitted), service assets (functionality the system provides), and infrastructure assets (components that support operation).

ID	Asset	Category	Description & Sensitivity
A-01	User credentials	Data — Critical	Username, bcrypt password hashes, email addresses stored in users.json. Compromise allows account takeover.
A-02	Encryption key	Infrastructure — Critical	Fernet symmetric key in data/secret.key. Compromise allows decryption of all stored documents and JSON files.
A-03	Uploaded documents	Data — High	Confidential files uploaded by users, stored as Fernet .enc files. Sensitivity varies by document content.
A-04	Session tokens	Data — High	256-bit URL-safe tokens in sessions.json. Theft allows session hijacking without credential compromise.
A-05	Reset tokens	Data — High	Password reset tokens in reset_tokens.json. Valid for 1 hour; single use. Theft allows account takeover.
A-06	Document metadata	Data — Medium	Filenames, owner IDs, share permissions, version history in documents.json. Leaks organisational relationships.
A-07	Security logs	Data — Medium	Structured JSON event log in security.log. Contains IP addresses, usernames, and action history. Privacy-sensitive.
A-08	Authentication service	Service — High	The login, registration, and password reset flows. Disruption prevents all user access.
A-09	Document sharing service	Service — Medium	Upload, download, and share functionality. Core business value of the application.
A-10	Flask application	Infrastructure — High	The running Python process and its configuration. Compromise gives code execution on the server.

B.2 Threat Enumeration

Threats are enumerated using STRIDE. Each threat maps to one or more assets and identifies the attack vector and attacker capability required.

Spoofing

ID	Threat	Asset	Risk	Description
T-01	Credential brute force	A-01	HIGH	Attacker submits many username/password combinations via /login. No prior knowledge required beyond a username.
T-02	Session token theft	A-04	HIGH	Attacker intercepts or extracts session_token cookie via network sniffing, XSS, or shared device access.
T-03	Password reset token theft	A-05	MEDIUM	Attacker intercepts reset link (e.g. email interception, server logs) and uses it to take over an account.
T-04	Username enumeration	A-01	MEDIUM	Attacker probes /login or /forgot-password to determine which usernames or emails are registered.

Tampering

ID	Threat	Asset	Risk	Description
----	--------	-------	------	-------------

T-05	Path traversal on upload	A-03, A-10	HIGH	Attacker crafts a filename such as ../../app.py or %2F%2E%2E%2Fetc%2Fpasswd to read or overwrite arbitrary files.
T-06	Stored XSS via document metadata	A-06	MEDIUM	Attacker injects JavaScript into a document description or filename that is later rendered in another user's browser.
T-07	Direct JSON file manipulation	A-01, A-03	LOW	Attacker with filesystem access edits users.json to elevate role or documents.json to grant share access.
T-08	Race condition on share grant	A-06	LOW	Concurrent share/revoke requests corrupt documents.json resulting in incorrect permissions.

Repudiation

ID	Threat	Asset	Risk	Description
T-09	Log tampering	A-07	MEDIUM	Attacker with filesystem access alters or deletes security.log to remove evidence of malicious activity.
T-10	Session-based action denial	A-07	LOW	User denies uploading a document or changing a share. Without append-only logs, attribution is difficult.

Information Disclosure

ID	Threat	Asset	Risk	Description
T-11	Encryption key exposure	A-02	CRITICAL	secret.key committed to version control or read by an unauthorised process allows full decryption of all data.
T-12	Insecure direct object reference	A-03	HIGH	Attacker guesses or enumerates document UUIDs and accesses /download/{id} without being granted share access.
T-13	Error message information leakage	A-10	MEDIUM	Unhandled exceptions expose stack traces, file paths, or internal state to the user via 500 error pages.
T-14	Traffic interception (HTTP)	A-01, A-04	HIGH	Credentials or session tokens transmitted over unencrypted HTTP intercepted by a network attacker.

Denial of Service

ID	Threat	Asset	Risk	Description
T-15	Login endpoint flooding	A-08	HIGH	High-volume login attempts exhaust server resources or lock out legitimate users via the account lockout mechanism.
T-16	Large file upload flood	A-09, A-10	MEDIUM	Attacker repeatedly uploads 16 MB files to exhaust disk space or memory.
T-17	Session JSON corruption	A-08	LOW	Corrupt sessions.json (e.g. via partial write) causes all session lookups to fail, logging out every user.

Elevation of Privilege

ID	Threat	Asset	Risk	Description
T-18	Horizontal privilege escalation	A-03	HIGH	User accesses another user's document by guessing its UUID at /download/{id} without being in the shares dict.
T-19	Vertical privilege escalation	A-01	HIGH	User manipulates role field in a request or exploits a missing role check to access /admin routes.
T-20	CSRF on state-changing routes	A-03, A-08	MEDIUM	Attacker tricks an authenticated user's browser into submitting a forged request to /delete or /share.

B.3 Vulnerability Assessment

Each vulnerability is assessed using a simplified risk matrix. Likelihood is scored 1–3 (low/medium/high). Impact is scored 1–3. Risk = Likelihood × Impact (max 9).

ID	Vulnerability	Likelihood	Impact	Score	Mitigation Status
V-01	Weak passwords accepted	High (3)	High (3)	9	MITIGATED — 12-char minimum + complexity regex enforced in <code>validate_password()</code>
V-02	Plaintext password storage	Med (2)	High (3)	6	MITIGATED — <code>bcrypt</code> cost 12; hash stored, never plaintext
V-03	Encryption key in version control	Med (2)	High (3)	6	MITIGATED — <code>.gitignore</code> excludes <code>secret.key</code> ; <code>chmod 600</code> on file
V-04	Path traversal in filename	High (3)	High (3)	9	MITIGATED — 4-stage <code>safe_filename_check()</code> including URL-decode + dot-segment rejection
V-05	Missing RBAC on routes	Med (2)	High (3)	6	MITIGATED — <code>@require_auth</code> and <code>@require_role</code> decorators on all protected routes
V-06	Insecure direct object reference	High (3)	High (3)	9	MITIGATED — <code>can_read()</code> checked before every download; 403 on failure
V-07	XSS via user input	Med (2)	Med (2)	4	MITIGATED — <code>html.escape()</code> on storage + Jinja2 auto-escaping on render
V-08	Missing security headers	Med (2)	Med (2)	4	MITIGATED — 7 headers set on every response in <code>after_request</code> hook
V-09	No account lockout	High (3)	High (3)	9	MITIGATED — 5-attempt lockout for 15 min; reset by admin via <code>/admin/unlock</code>
V-10	Concurrent session overwrite	High (3)	Med (2)	6	MITIGATED — <code>fcntl.LOCK_EX</code> on all JSON writes; <code>throttled touch()</code>
V-11	CSRF on state-changing requests	Med (2)	Med (2)	4	PARTIAL — <code>SameSite=Strict</code> on cookies mitigates most CSRF; no explicit CSRF token
V-12	Stack trace in error pages	Med (2)	Med (2)	4	MITIGATED — custom 403/404/413/500 handlers return generic messages

B.4 Attack Scenarios

Scenario 1 — Credential Brute Force leading to Account Takeover

Field	Detail
Attacker goal	Gain access to a user account without knowing the correct password
Preconditions	Attacker knows a valid username (e.g. from the registration page error messages or a leaked list)
Attack steps	1. Submit POST <code>/login</code> with known username and dictionary passwords at high rate. 2. After 5 failures, account locks for 15 min. 3. Attacker rotates to a different IP to bypass per-IP rate limit.
Controls triggered	<code>check_rate_limit()</code> blocks >10 attempts/IP/min. <code>authenticate_user()</code> locks the account after 5 failures regardless of IP. <code>bcrypt</code> cost 12 adds ~250ms per attempt, making large-scale brute force economically impractical.
Residual risk	Distributed attacks across many IPs can still lock accounts (DoS against specific users). The in-memory rate limiter resets on server restart.

Scenario 2 — Path Traversal on Document Upload

Field	Detail
-------	--------

Attacker goal	Overwrite a sensitive server file (e.g. config.py or secret.key) by uploading with a crafted filename
Preconditions	Attacker holds a valid user account with upload permission (not a guest)
Attack steps	1. Submit POST /upload with filename: ../../config.py or URL-encoded %2E%2E%2Fconfig.py. 2. Application writes encrypted content to the traversed path.
Controls triggered	safe_filename_check() step 2 URL-decodes the filename before any check. Step 3 rejects any path segment composed entirely of dots. werkzeug.secure_filename() provides a final normalisation. os.path.abspath() secondary check in download_document() confirms path is inside UPLOAD_DIR.
Residual risk	Low. Four independent layers must all fail simultaneously for this attack to succeed.

Scenario 3 — Insecure Direct Object Reference on Document Download

Field	Detail
Attacker goal	Download a document owned by another user without being granted share access
Preconditions	Attacker is authenticated and has guessed or enumerated a valid document UUID
Attack steps	1. Observe /download/abc-123 pattern from own documents. 2. Enumerate other UUID values. 3. Attempt GET /download/{guessed-id}.
Controls triggered	download_document() calls can_read(doc, user_id, user_role) before any I/O. If the user is not the owner, admin, or in the shares dict, the function returns (None, 'Access denied.') and the route aborts with 403. The attempt is logged as ACCESS_DENIED.
Residual risk	Low. UUIDs are not sequential (128-bit random), so enumeration is computationally infeasible. The 403 response reveals nothing about the document's existence.

Scenario 4 — Admin Account Compromise via Weak Password

Field	Detail
Attacker goal	Gain admin role to access all documents and manage user accounts
Preconditions	Attacker knows the admin username and the admin has registered with a weak password
Attack steps	1. Attempt login with common passwords (password123, admin2026, etc.). 2. If successful, gain access to /admin.
Controls triggered	validate_password() enforces 12+ chars with uppercase, lowercase, digit, and special character at registration — weak passwords cannot be registered. Account lockout triggers after 5 failures. bcrypt cost 12 limits guessing speed.
Residual risk	Medium. The first admin account is created by manually editing users.json — if this step is done carelessly (e.g. setting a simple test password), the policy is bypassed.

B.5 Risk Prioritization

Threats are ranked by their overall risk score and grouped into response tiers. Mitigated items are included to document the controls in place.

Ran k	Threat	Risk	Status	Priority Action
1	T-11 — Encryption key exposure (A-02)	CRITICAL	Mitigated	.gitignore + chmod 600 + never log key path
2	T-05 — Path traversal on upload (A-03)	HIGH	Mitigated	4-layer filename sanitisation pipeline
3	T-12 — IDOR on document download (A-03)	HIGH	Mitigated	can_read() before every I/O + 403 + logging
4	T-01 — Credential brute force (A-01)	HIGH	Mitigated	Rate limit + lockout + bcrypt cost 12
5	T-14 — Traffic interception (A-01, A-04)	HIGH	Mitigated	TLS + HSTS + Secure cookie flag
6	T-18/T-19 — Privilege escalation (A-03)	HIGH	Mitigated	RBAC decorators + role check in every route

7	T-15 — Login flood DoS (A-08)	HIGH	Partial	IP rate limit; consider CAPTCHA for production
8	T-20 — CSRF (A-03, A-08)	MEDIUM	Partial	SameSite=Strict cookies; add CSRF token in production
9	T-04 — Username enumeration (A-01)	MEDIUM	Mitigated	Generic error messages on login + forgot-password
10	T-09 — Log tampering (A-07)	MEDIUM	Residual	Append-only log mode or external SIEM in production

Section C — Security Controls

For each of the seven security requirements from the project specification, this section documents the control description, implementation approach, testing methodology, known limitations, and mitigation strategies.

C.1 User Authentication (15 points)

Aspect	Detail
Control description	Secure credential registration with input validation, bcrypt password hashing, account lockout after 5 failed attempts, per-IP rate limiting at 10 attempts per 60-second window, and session creation on successful login.
Implementation	Registration: <code>validate_username()</code> enforces <code>[w]{3,20}</code> regex; <code>validate_email()</code> enforces RFC pattern; <code>validate_password()</code> enforces 12-char minimum with complexity; duplicate username/email rejected before writing. Hashing: <code>bcrypt.hashpw(password.encode(), bcrypt.gensalt(rounds=12))</code> . Cost factor 12 produces ~250ms hash time on modern hardware. Hash stored as <code>\$2b\$12\$...</code> string — plaintext never persisted. Lockout: <code>failed_attempts</code> counter incremented on each bcrypt mismatch. At <code>MAX_FAILED_ATTEMPTS</code> (5), <code>locked_until = time.time() + LOCKOUT_DURATION</code> (900s). Admin can reset via <code>/admin/users/{id}/unlock</code> . Rate limit: In-memory dict keyed by IP; old timestamps pruned on each call. Returns False at 10+ attempts within <code>RATE_LIMIT_WINDOW</code> (60s).
Testing methodology	88-test unittest suite includes: <code>test_missing_uppercase/lowercase/digit/special_char</code> , <code>test_too_short</code> , <code>test_account_locks_after_5_failures</code> , <code>test_correct_password_rejected_while_locked</code> , <code>test_error_does_not_reveal_username_existence</code> , <code>test_eleventh_attempt_blocked</code> , <code>test_different_ips_independent</code> , <code>test_bcrypt_cost_factor_is_12</code> .
Known limitations	(1) Rate limiter is in-memory — resets on server restart and does not share state across multiple processes. (2) First admin account is created by manually editing <code>users.json</code> , bypassing password validation. (3) No CAPTCHA — a distributed attacker can probe from many IPs.
Mitigation strategies	For production: use a persistent rate limit store (Redis); generate admin via a management command that runs <code>validate_password()</code> ; add CAPTCHA or proof-of-work on the login endpoint.

C.2 Access Control (15 points)

Aspect	Detail
Control description	Role-Based Access Control (RBAC) with three roles: admin (full access), user (standard), guest (read-only on shared content). Document-level permissions (owner, editor, viewer) applied independently of system role.
Implementation	<code>@require_auth</code> decorator checks <code>g.user</code> is not None (set by <code>load_session()</code>); redirects to <code>/login</code> if absent. <code>@require_role(*roles)</code> calls <code>abort(403)</code> if <code>g.user['role']</code> not in the allowed set. Document permission functions <code>can_read()</code> , <code>can_write()</code> , <code>can_delete()</code> are pure functions called before any I/O — admin bypasses all; owner can do everything; editor can read/write; viewer can only read.
Testing methodology	16 access control tests: <code>test_owner_can_read/write/delete</code> , <code>test_editor_can_write</code> , <code>test_viewer_cannot_write/delete</code> , <code>test_stranger_cannot_read/write/delete</code> , <code>test_admin_can_read/write/delete_any</code> , <code>test_guest_cannot_read_unshared</code> . All using mock doc dicts — no HTTP required.
Known limitations	(1) Role is stored in <code>users.json</code> alongside the password hash — a local attacker with file access can escalate privileges without detection. (2) No fine-grained permission expiry — a shared viewer stays a viewer indefinitely until manually revoked. (3) CSRF tokens are not implemented; <code>SameSite=Strict</code> provides partial mitigation.
Mitigation strategies	Store roles in a separate signed JWT or append-only role-change log to detect tampering. Add share expiry timestamps (e.g. <code>expires_at</code> field in the <code>shares</code> dict). Implement Flask-WTF CSRF tokens for production.

C.3 Input Validation & Injection Prevention (20 points)

Aspect	Detail
--------	--------

Control description	Whitelist validation on all inputs; HTML escaping on all outputs; four-stage filename sanitisation; length caps on all fields; MIME/extension whitelist on uploads.
Implementation	XSS prevention: sanitize() calls html.escape() on all user input before storage. Jinja2 auto-escaping provides a second layer at render time. {{ var safe }} is never used in any template. Path traversal: safe_filename_check() pipeline: null-byte rejection → URL-decode (unquote) → dot-segment rejection → werkzeug.secure_filename → whitelist regex [w\-\.\+]. Secondary os.path.abspath() guard on download. Length limits: Username 20 chars, email 254 chars (RFC 5321), password 128 chars, description 500 chars, MAX_CONTENT_LENGTH 16 MB enforced by Flask before route handler runs.
Testing methodology	XSS tests: test_script_tag_escaped (asserts <script> in output), test_img_onerror_escaped, test_javascript_uri_escaped, test_ampersand_escaped. Path traversal tests: test_dotdot_slash_rejected, test_windows_traversal_rejected, test_null_byte_rejected, test_absolute_path_rejected, test_encoded_slash_rejected, test_hidden_traversal_rejected.
Known limitations	(1) MIME type verification uses only file extension — no magic-byte or content-type sniffing against the actual file content. (2) No malware scanning of uploaded files. (3) Command injection is not applicable (no shell commands are executed), but is noted as a non-issue here.
Mitigation strategies	For production: add python-magic library to verify file magic bytes against declared extension. Integrate a ClamAV scan on upload (clamd Python binding). Consider a Content-Disposition: attachment header on all downloads to prevent MIME confusion attacks.

C.4 Encryption (15 points)

Aspect	Detail
Control description	Transport encryption via TLS with HSTS enforcement; data-at-rest encryption using Fernet (AES-128-CBC + HMAC-SHA256) for both uploaded files and sensitive JSON data.
Implementation	Transport: Flask ssl_context=('cert.pem','key.pem') for dev; force_https() before_request redirects HTTP with 301; HSTS header max-age=31536000; includeSubDomains on every response. At-rest: EncryptedStorage.encrypt_file() reads raw bytes, calls Fernet.encrypt(), writes .enc; decrypt_file() reverses. Sensitive JSON (users, sessions, reset tokens) uses save()/load() with the same cipher.
Testing methodology	Encryption tests: test_encrypted_file_differs_from_plaintext (asserts secret not in raw bytes), test_round_trip_preserves_data, test_key_file_permissions_are_owner_only (asserts oct mode == '600'), test_encrypted_file_binary_not_json (json.loads raises), test_different_key_cannot_decrypt (raises InvalidToken), test_load_missing_file_returns_default. HTTP header test: verified via Flask test client that all 7 headers present.
Known limitations	(1) Fernet uses a single symmetric key for all data — key compromise exposes everything. (2) Self-signed certificate in dev mode will trigger browser warnings. (3) Key rotation requires re-encrypting all files. (4) No envelope encryption (i.e. no per-document keys wrapped by a master key).
Mitigation strategies	For production: use a CA-signed certificate (Let's Encrypt). Implement envelope encryption: generate a random per-document key, encrypt the document with it, then encrypt that key with the master Fernet key — stored alongside the document metadata. Integrate a KMS (AWS KMS, HashiCorp Vault) for master key management.

C.5 Session Management (15 points)

Aspect	Detail
Control description	Custom token-based sessions using 256-bit cryptographically random tokens, stored in a JSON file with exclusive file locking. 30-minute inactivity timeout. HttpOnly, SameSite=Strict, Secure cookie attributes. Session invalidation on logout and password change.
Implementation	SessionManager.create() calls secrets.token_urlsafe(32). Token and metadata (user_id, created_at, last_activity, ip, user_agent) written under fcntl.LOCK_EX. validate() reads only (no write); only acquires write lock to remove expired sessions. touch() refreshes last_activity at most once per 60 seconds. destroy_all_for_user() called on password change and admin reset. Cookie: httponly=True, secure=(not DEBUG), samesite='Strict', max_age=1800.
Testing methodology	9 session tests: test_create_returns_token, test_token_is_url_safe_random (10 unique tokens), test_valid_token_resolves_to_session, test_invalid_token_returns_none, test_expired_session_rejected (forced last_activity to past), test_destroy_removes_session, test_multiple_sessions_coexist (validates tok_a

	after tok_b created), test_destroy_all_for_user_removes_only_their_sessions, test_purge_expired_removes_only_old_sessions.
Known limitations	(1) Session store is a flat JSON file — not suitable for high concurrency. (2) Secure=False in dev mode means the session cookie can be sent over HTTP during local testing. (3) No concurrent session limit per user. (4) last_activity throttle means up to 60 seconds of extra session lifetime on expiry.
Mitigation strategies	For production: migrate session store to Redis with atomic operations (SET NX with TTL). Add a max concurrent sessions limit per user (e.g. 3). Set Secure=True unconditionally (requires TLS everywhere). Consider binding sessions to IP + User-Agent for additional hijacking resistance.

C.6 Security Headers (10 points)

Aspect	Detail
Control description	Seven security headers applied to every HTTP response via the set_security_headers() after_request hook. Headers defend against XSS, clickjacking, MIME sniffing, traffic downgrade, and browser feature abuse.
Implementation	Content-Security-Policy: default-src 'self'; script-src 'self' 'unsafe-inline' cdn.jsdelivr.net; style-src 'self' 'unsafe-inline' cdn.jsdelivr.net; img-src 'self' data:; font-src 'self' cdn.jsdelivr.net; connect-src 'self'; frame-ancestors 'none' X-Frame-Options: DENY — prevents all iframe embedding (clickjacking) X-Content-Type-Options: nosniff — prevents MIME sniffing of response bodies X-XSS-Protection: 1; mode=block — legacy IE/early Chrome XSS filter Referrer-Policy: strict-origin-when-cross-origin — limits referrer leakage Permissions-Policy: geolocation=(), microphone=(), camera=() — blocks sensor APIs Strict-Transport-Security: max-age=31536000; includeSubDomains — HSTS 1-year pin
Testing methodology	Flask test client GET /login verified for all 7 headers using automated assertion: response.headers.get(header) checked for all required substrings. Verified in CI via the test_security.py suite. Manual verification via https://securityheaders.com when running with TLS.
Known limitations	(1) 'unsafe-inline' in CSP weakens script-src and style-src protection — any XSS that injects inline scripts bypasses this directive. (2) HSTS only takes effect after the first HTTPS visit — first-visit MITM downgrade is still possible without HSTS preloading. (3) X-XSS-Protection is deprecated in modern browsers.
Mitigation strategies	Replace 'unsafe-inline' with cryptographic nonces (generate a random nonce per request; inject into script tags; include in CSP header). Submit domain to the HSTS preload list (hstspreload.org) to eliminate first-visit vulnerability. Remove X-XSS-Protection (deprecated) and rely on CSP alone.

C.7 Logging & Monitoring (10 points)

Aspect	Detail
Control description	A module-level SecurityLogger singleton writes structured JSON events to logs/security.log. Every security-relevant action is logged with timestamp, event type, severity, user ID, IP address, user agent, and details.
Implementation	SecurityLogger wraps Python's standard logging module with a FileHandler. log_event() serialises a dict to JSON and calls the appropriate logging level. Convenience methods (login_success, login_failed, account_locked, access_denied, data_access, etc.) ensure consistent field names. Imported as security_log singleton in app.py, auth.py, and documents.py.
Events logged	LOGIN_SUCCESS · LOGIN_FAILED · ACCOUNT_LOCKED · LOGOUT · REGISTRATION · ACCESS_DENIED · DATA_ACCESS · SESSION_CREATED · SESSION_EXPIRED · INPUT_VALIDATION_FAILED · RATE_LIMIT_EXCEEDED · SUSPICIOUS_ACTIVITY · PASSWORD_CHANGED · PASSWORD_RESET_REQUESTED · DOCUMENT_SHARED · DOCUMENT_DELETED · ROLE_CHANGED · ADMIN_PASSWORD_RESET
Testing methodology	Logging is verified implicitly via integration: authenticated routes call security_log methods which write to the file. Direct unit testing of log output is omitted from the test suite to avoid log-file side effects — the logging module itself is trusted. Manual inspection of security.log after running test scenarios confirms correct JSON structure.

Known limitations	(1) Log file is writable by the same process that serves the application — a compromise allows log tampering. (2) No log rotation — security.log grows unbounded. (3) No real-time alerting — logs must be manually reviewed. (4) IP addresses may be unreliable behind proxies unless X-Forwarded-For is handled.
Mitigation strategies	For production: write logs to a remote syslog or SIEM (e.g. ELK stack, Splunk). Use Python's RotatingFileHandler for automatic rotation. Add alerting rules for CRITICAL events (ACCOUNT_LOCKED, RATE_LIMIT_EXCEEDED, SUSPICIOUS_ACTIVITY). Trust X-Forwarded-For only from known proxy IPs (Werkzeug ProxyFix middleware).

Section D — Data Protection

This section documents how sensitive data is classified, how it is protected at rest and in transit, how the encryption key is managed, and what procedures exist for secure deletion of data.

D.1 Data Classification

Data is classified into four tiers based on sensitivity. The classification determines the required protection level and governs access, storage, and deletion procedures.

Tier	Class	Examples	Protection requirements
T1	Critical	Fernet secret.key, bcrypt password hashes	Never transmitted. Stored with chmod 600. Not logged. Excluded from version control via .gitignore. key must be backed up offline.
T2	Confidential	Uploaded documents (.enc files), session tokens, reset tokens	Encrypted at rest (Fernet). Transmitted over TLS only. Session tokens in HttpOnly cookies. Reset tokens single-use, 1-hour expiry. File contents never logged.
T3	Internal	User accounts (username, email, role), document metadata, share permissions	Stored in Fernet-encrypted JSON. Accessible only to authenticated users scoped to their own data (or admin). Email addresses treated as PII — not displayed to other users.
T4	Public / Operational	Error messages, flash notifications, security log event types	Generic error messages only — no internal paths, stack traces, or user details in responses. Security logs stored server-side only and not exposed via any route.

D.2 Encryption Methods

Two distinct encryption mechanisms are used: password hashing for credentials, and symmetric authenticated encryption for all other sensitive data.

Password Hashing — bcrypt

Property	Detail
Algorithm	bcrypt (Blowfish-based, Eksblowfish key schedule)
Cost factor	12 (configurable via Config — currently ~250ms per verification on modern hardware)
Salt	Automatically generated per-password by <code>bcrypt.gensalt(rounds=12)</code> — 128-bit random, encoded in the hash string
Output format	<code>\$2b\$12\${22-char-salt}\${31-char-hash}</code> — 60-character ASCII string stored in <code>users.json</code>
Verification	<code>bcrypt.checkpw(submitted.encode(), stored_hash.encode())</code> — constant-time comparison prevents timing attacks
Why not Argon2	bcrypt is the course-recommended library and is battle-tested. Argon2 would be preferred for new production systems (winner of Password Hashing Competition) but requires <code>argon2-cffi</code> — not installed. bcrypt at cost 12 is adequate for this use case.

Data-at-Rest Encryption — Fernet (AES-128-CBC + HMAC-SHA256)

Property	Detail
Algorithm	AES-128-CBC for confidentiality + HMAC-SHA256 for integrity and authenticity
Key size	256-bit total: 128-bit AES key + 128-bit HMAC key derived from the stored Fernet key via HKDF
IV	128-bit IV generated fresh per encryption operation by <code>os.urandom</code> — never reused
Format	Version byte (0x80) Timestamp (8 bytes) IV (16 bytes) Ciphertext (padded) HMAC (32 bytes) — all base64url-encoded
Scope	Applied to: uploaded document files (.enc), <code>users.json</code> , <code>sessions.json</code> , <code>documents.json</code> , <code>reset_tokens.json</code>

Authentication	HMAC verification performed before decryption — tampered ciphertext raises <code>cryptography.fernet.InvalidToken</code> without any plaintext being returned
----------------	---

Transport Encryption — TLS

Property	Detail
Development	Self-signed certificate: <code>openssl req -x509 -newkey rsa:4096 -nodes -out cert.pem -keyout key.pem -days 365</code> . <code>Flask ssl_context=('cert.pem','key.pem')</code> .
Production	CA-signed certificate via Let's Encrypt (certbot). Unicorn with <code>--certfile</code> and <code>--keyfile</code> flags behind an nginx reverse proxy.
Enforcement	<code>force_https()</code> before_request hook issues 301 redirect for all HTTP requests. HSTS header prevents future HTTP connections for 1 year.

D.3 Key Management

The Fernet symmetric key is the most sensitive asset in the system. Its lifecycle — generation, storage, access, rotation, and backup — is governed by the following procedures.

Key Generation

- Generated automatically by `EncryptedStorage.__init__()` on first run using `Fernet.generate_key()`, which calls `os.urandom(32)` internally.
- Written to `data/secret.key` on first startup. If the file already exists, it is read — no new key is generated.
- File permissions set to `0o600` (owner read/write only) immediately after write via `os.chmod(key_file, 0o600)`.

Key Storage

- Stored at `data/secret.key` — a path listed in `.gitignore` to prevent accidental version control commit.
- Not stored in environment variables (would appear in process list) or `config.py` (would be committed).
- In production, `secret.key` should be stored on a separate encrypted volume or in a secrets manager (AWS Secrets Manager, HashiCorp Vault) and injected at runtime.

Key Access

- Read once at application startup by `EncryptedStorage.__init__()`. The key material is held in memory as `self.key` and used by `self.cipher` (Fernet instance).
- Never logged, never included in error messages, never transmitted.
- The key path (`Config.KEY_FILE`) is used only by `EncryptedStorage` — no other module accesses the key file directly.

Key Rotation

- No automated rotation is implemented (beyond scope for this project).
- Manual rotation procedure: (1) Generate a new Fernet key. (2) Decrypt all encrypted files using the old key. (3) Re-encrypt all files with the new key. (4) Replace `secret.key`. (5) Destroy the old key securely. (6) Verify integrity of all re-encrypted files before destroying the old key backup.
- Fernet MultiFernet can be used to support dual-key operation during rotation — old key decrypts, new key re-encrypts, old key then removed.

Key Backup

- The key must be backed up before deploying, otherwise all encrypted data becomes permanently unreadable if the key file is lost.
- Backup should be encrypted with a separate passphrase (e.g. GPG symmetric encryption) and stored offline (separate physical location from the server).
- Backup copies must have the same `0o600` permissions and never be transmitted over insecure channels.

D.4 Secure Deletion Procedures

Secure deletion ensures that removed data cannot be recovered from disk, memory dumps, or residual storage. The following procedures apply to each data type.

User Account Deletion

- No explicit account deletion route exists in the current implementation (out of scope per project spec).
- To delete a user manually: remove the entry from `users.json`, call `session_mgr.destroy_all_for_user(user_id)` to invalidate all active sessions, and remove the user from all document share dicts.
- For production: implement a `DELETE /users/{id}` route gated behind `@require_role('admin')` with a confirmation step and event logging (`USER_DELETED`, `severity=WARNING`).

Document Deletion

- `delete_document()` removes all versioned `.enc` files for the document using `os.remove(enc_path)`, then removes the document metadata entry from `documents.json` under `fcntl` exclusive lock.
- Limitation: `os.remove()` unlinks the directory entry but does not overwrite the file contents on disk. On SSDs with wear levelling, the data may persist in unmapped flash pages until overwritten by the drive controller.
- For production: overwrite file contents with random bytes before unlinking — e.g. open file in write mode, write `os.urandom(file_size)` bytes, close, then `os.remove()`. Python's `shutil` does not provide secure deletion natively.

Session Deletion

- `SessionManager.destroy(token)` removes the token from `sessions.json` under `fcntl` lock. The entry is fully deleted (not marked inactive).
- `destroy_all_for_user(user_id)` iterates all sessions and removes any belonging to the specified user — called on password change and admin password reset.
- `purge_expired()` removes all sessions whose `last_activity` is older than `SESSION_TIMEOUT` — can be called on a schedule or at login time to prevent unbounded growth.

Reset Token Deletion

- Consumed reset tokens are marked `used=true` rather than deleted — this allows replay attempts to be detected and rejected with a specific error message ('This reset link has already been used.').
- Old tokens for a user are deleted when a new reset is requested for the same email, preventing token accumulation.
- For production: schedule a purge job that deletes tokens where `expires_at < time.time() - 86400` (older than 24h post-expiry) to prevent indefinite retention of expired tokens in `reset_tokens.json`.

Encryption Key Destruction

- If `secret.key` must be destroyed (e.g. system decommission), use a secure file deletion tool: `shred -u data/secret.key` on Linux (overwrites with random bytes multiple times before unlinking).
- After key destruction, all encrypted data (`.enc` files, encrypted JSON) becomes permanently unreadable. Verify this is the intended outcome before destroying the key.
- On SSDs, shred effectiveness is not guaranteed due to wear levelling — consider full-disk encryption (LUKS, FileVault) as the primary protection mechanism, with the application-level Fernet key providing defence-in-depth.